

Applications of Parallel Computing

S. Afkhami

Applications of Parallel Computing

Copyright (C) 2023 S. Afkhami

Faculty Authored Open Textbook
New Jersey Institute of Technology
Newark, NJ, USA

Permission is granted to copy, distribute, transmit and adapt this work under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License: <https://creativecommons.org/licenses/by-nc/4.0/>.

If you are interested in distributing a commercial version of this work, please contact the author.

The L^AT_EX source and code for this book is available from
<https://github.com/afkhamis/HPC-book>

CONTENTS

1	Introduction to Parallel Computing	1
1.1	Serial Computing	1
1.2	Parallel Computing Overview	3
1.2.1	Shared Memory Model	6
1.2.2	Distributed Memory Model	7
1.3	Case study: Matrix Multiplication	8
2	Programming Tools	9
2.1	Command-line Environments	9
2.2	Integrated Development Environments (IDE) and Text Editors . .	13
2.3	Compilers and makefiles	13
2.4	Debugging	16
2.5	Version control	17
3	C Programming	21
3.1	The first program	21
3.2	Variables	23
3.3	Inputting and Outputting	24
3.4	Operations	28
3.5	Boolean Algebra	29
3.6	Mathematical Functions	31
3.7	Functions	31
3.8	Glossary	34
4	Shared Memory Parallelism	35
4.1	What Is Shared?	35
4.2	Memory Modules	36

4.2.1	Interleaving	36
4.2.2	Bank Conflicts and Solutions	37
4.2.3	Example: Code to Implement Padding	39
4.3	Interconnection Topologies	40
4.3.1	SMP Systems	40
4.3.2	NUMA Systems	41
4.3.3	NUMA Interconnect Topologies	41
4.3.4	Comparative Analysis	44
4.3.5	Why Have Memory in Modules?	45
4.4	Synchronization Hardware	46
4.4.1	Test-and-Set Instructions	46
4.4.2	May Not Need the Latest	48
4.4.3	Fetch-and-Add Instructions	49
4.5	Cache Issues	50
4.5.1	Cache Coherency	50
4.5.2	Example: the MESI Cache Coherency Protocol	53
4.5.3	The Problem of “False Sharing”	55
4.6	Memory-Access Consistency Policies	55
4.7	Fetch-and-Add Combining within Interconnects	58
4.8	Multicore Chips	58
4.9	Optimal Number of Threads	59
4.10	Processor Affinity	60
4.11	Illusion of Shared-Memory through Software	60
4.11.1	Software Distributed Shared Memory	60
4.12	Barrier Implementation	62
4.12.1	A Use-Once Version	63
4.12.2	An Attempt to Write a Reusable Version	63
4.12.3	A Correct Version	64
4.12.4	Refinements	65
5	Parallel Computing with OpenMP	69
5.1	The OpenMP programming model	70
5.1.1	Critical section	71
5.2	How to use OpenMP	72
5.3	π — OpenMP style	76
5.4	Compiling and running an OpenMP application	77
5.5	Final remarks	77
5.6	Further reading	78
6	Introduction to OpenMP	79
6.1	Overview	79

6.2	Example: Dijkstra Shortest-Path Algorithm	79
6.2.1	The Algorithm	82
6.2.2	The OpenMP <code>parallel</code> Pragma	82
6.2.3	Scope Issues	83
6.2.4	The OpenMP <code>single</code> Pragma	84
6.2.5	The OpenMP <code>barrier</code> Pragma	84
6.2.6	Implicit Barriers	84
6.2.7	The OpenMP <code>critical</code> Pragma	85
6.3	The OpenMP <code>for</code> Pragma	85
6.3.1	Example: Dijkstra with Parallel <code>for</code> Loops	85
6.3.2	Nested Loops	87
6.3.3	Controlling the Partitioning of Work to Threads: the <code>sched-</code> <code>ule</code> Clause	88
6.3.4	Example: In-Place Matrix Transpose	90
6.3.5	The OpenMP <code>reduction</code> Clause	91
6.4	Example: Mandelbrot Set	92
6.5	The Task Directive	96
6.5.1	Example: Quicksort	96
6.6	Other OpenMP Synchronization Issues	97
6.6.1	The OpenMP <code>atomic</code> Clause	97
6.6.2	Memory Consistency and the <code>flush</code> Pragma	99
6.7	Combining Work-Sharing Constructs	100
6.8	The Rest of OpenMP	100
6.9	Compiling, Running and Debugging OpenMP Code	100
6.9.1	Compiling	100
6.9.2	Running	101
6.9.3	Debugging	101
6.10	Performance	102
6.10.1	The Effect of Problem Size	102
6.10.2	Some Fine Tuning	102
6.10.3	OpenMP Internals	106
6.11	Example: Root Finding	107
6.12	Example: Mutual Outlinks	109
6.13	Example: Transforming an Adjacency Matrix	110
6.14	Example: Finding the Maximal Burst in a Time Series	113
6.15	Locks with OpenMP	116
6.16	Other Examples of OpenMP Code in This Book	117
7	Message Passing Systems	119
7.1	Overview	119
7.2	A Historical Example: Hypercubes	120

7.2.1	Definitions	120
7.3	Networks of Workstations (NOWs)	122
7.3.1	The Network Is Literally the Weakest Link	122
7.3.2	Other Issues	123
7.4	Scatter/Gather Operations	124
8	Distributed Programming with MPI	125
8.1	Introduction to MPI	125
8.2	Message Passing	125
8.2.1	Communication Routines	125
8.2.2	Collective Communications	125
8.3	Performance	125
9	Introduction to MPI	127
9.1	Overview	127
9.1.1	History	127
9.1.2	Structure and Execution	128
9.1.3	Implementations	128
9.1.4	Performance Issues	128
9.2	Review of Earlier Example	129
9.3	Example: Dijkstra Algorithm	129
9.3.1	The Algorithm	129
9.3.2	The MPI Code	130
9.3.3	Introduction to MPI APIs	133
9.4	Example: Removing 0s from an Array	135
9.5	Debugging MPI Code	138
9.6	Collective Communications	138
9.6.1	Example: Refined Dijkstra Code	138
9.6.2	MPI_Bcast()	141
9.6.3	MPI_Reduce()/MPI_Allreduce()	142
9.6.4	MPI_Gather()/MPI_Allgather()	143
9.6.5	The MPI_Scatter()	144
9.6.6	Example: Count the Number of Edges in a Directed Graph	144
9.6.7	Example: Cumulative Sums	144
9.6.8	Example: an MPI Solution to the Mutual Outlinks Problem	146
9.6.9	The MPI_Barrier()	148
9.6.10	Creating Communicators	149
9.7	Buffering, Synchrony and Related Issues	149
9.7.1	Buffering, Etc.	150
9.7.2	Safety	151
9.7.3	Living Dangerously	152

9.7.4 Safe Exchange Operations	152
10 Introduction to Parallel Sorting	153
10.1 Quicksort	153
10.1.1 The Separation Process	153
10.1.2 Example: OpenMP Quicksort	155
10.1.3 Hyperquicksort	156
10.2 Mergesorts	157
10.2.1 Sequential Form	157
10.2.2 Shared-Memory Mergesort	157
10.2.3 Message Passing Mergesort on a Tree Topology	157
10.2.4 Compare-Exchange Operations	158
10.2.5 Bitonic Mergesort	158
10.3 The Bubble Sort and Its Cousins	160
10.3.1 The Much-Maligned Bubble Sort	160
10.3.2 A Popular Variant: Odd-Even Transposition	161
10.3.3 Example: CUDA Implementation of Odd/Even Transposi- tion Sort	161
10.4 Shearsort	162
10.5 Bucket Sort with Sampling	163
10.6 Radix Sort	167
10.7 Enumeration Sort	167
11 GPU computing with CUDA	169
11.1 What is GPU?	169
11.2 What is the difference between GPU and CPU?	170
11.3 What is the advantage of using GPU for computation?	171
11.4 What are the important parts inside a GPU?	171
11.5 How does CUDA connect with hardware?	173
11.6 Is CUDA the only GPU programming language available?	174
11.7 Introduction to CUDA	174
11.8 Acknowledgement	175
11.9 An Example of Vector Addition	175
11.9.1 The Device Code	175
11.9.2 The Host Code	176
11.9.3 Compiling the code and executing this example	181
11.10 Vector Addition with Blocks	182
11.10.1 Block	182
11.10.2 The Device Code	183
11.10.3 The Host Code	184
11.11 Vector Addition with Blocks and Threads	184

11.11.1 Threads	184
11.11.2 The Device Code	185
11.11.3 The Host Code	185
12 Thread Advance	187
12.1 Acknowledgement	187
12.2 Vector Dot Product	187
12.2.1 The Device Code	187
12.2.2 The Host Code	188
12.3 Vector Dot Product with Reduction	190
12.3.1 The Device Code	191
12.3.2 The Host Code	194
13 CUDA in Two-dimension	197
13.1 Acknowledgement	197
13.2 An Example of Matrix Multiplication	197
13.2.1 Performance	197
13.3 Global Memory Version	198
13.3.1 The Device Code	198
13.3.2 The Host Code	199
13.3.3 Performance	201
13.4 CUDA Memory Types	202
13.5 Shared Memory Version	203
13.5.1 The Device Code	205
13.5.2 About blockDim and matrix dimension	207
13.5.3 The Host Code	207
13.5.4 Performance	210
14 Introduction to GPU Programming with CUDA	211
14.1 Overview	211
14.2 Some Terminology	212
14.3 Example: Calculate Row Sums	212
14.4 Understanding the Hardware Structure	217
14.4.1 Processing Units	217
14.4.2 Thread Operation	217
14.4.3 Memory Structure	219
14.4.4 Threads Hierarchy	226
14.4.5 What's NOT There	228
14.5 Synchronization, Within and Between Blocks	228
14.6 More on the Blocks/Threads Tradeoff	230
14.7 Hardware Requirements, Installation, Compilation, Debugging	231

14.8 Example: Improving the Row Sums Program	233
14.9 Example: Finding the Mean Number of Mutual Outlinks	235
14.10 Example: Finding Prime Numbers	237
14.11 Example: Finding Cumulative Sums	241
14.12 When Is It Advantageous to Use Shared Memory	243
14.13 Example: Transforming an Adjacency Matrix	243
14.14 Error Checking	247
14.15 Loop Unrolling	247
14.16 Short Vectors	248
14.17 Newer Generations	248
14.17.1 True Caching	248
14.17.2 Unified Memory	248
14.18 CUDA from a Higher Level	249
14.18.1 CUBLAS	249
14.18.2 Thrust	252
14.18.3 CUFFT	252
14.19 Other CUDA Examples in This Book	252
15 Introduction to OpenCL model	253
15.1 OpenCL model and components	253
15.2 Device architecture	253
15.2.1 Device management and kernel Launch	253
15.3 Synchronization on OpenCL	253
Index	255
15.4 Parallel CUDA	258
15.4.1 Memory management	258
15.5 Atomic	258
15.6 Streams	258
16 Numerical Methods	259
16.1 Finite Difference Methods	259
16.2 Pseudospectral Methods	259
16.3 Linear Solvers and Libraries	259
17 Fast Fourier Transform	261
17.1 The Serial Algorithm	261
17.2 The Cooley-Tukey algorithm	261
17.3 Parallel Fast Fourier Transform and fftw	261
18 Parallel Computation for Audio and Image Processing	263
18.1 General Principles	263

18.1.1 One-Dimensional Fourier Series	263
18.1.2 Two-Dimensional Fourier Series	266
18.2 Discrete Fourier Transforms	267
18.2.1 One-Dimensional Data	267
18.2.2 Inversion	268
18.2.3 Two-Dimensional Data	269
18.3 Parallel Computation of Discrete Fourier Transforms	270
18.3.1 The Fast Fourier Transform	270
18.3.2 A Matrix Approach	270
18.3.3 Parallelizing Computation of the Inverse Transform	271
18.3.4 Parallelizing Computation of the Two-Dimensional Transform	271
18.4 Available FFT Software	272
18.4.1 R	272
18.4.2 CUFFT	272
18.4.3 FFTW	272
18.5 Applications to Image Processing	272
18.5.1 Smoothing	273
18.5.2 Example: Audio Smoothing in R	273
18.5.3 Edge Detection	274
18.6 R Access to Sound and Image Files	275
18.7 Keeping the Pixel Intensities in the Proper Range	275
18.8 Does the Function <code>g()</code> Really Have to Be Repeating?	275
18.9 Vector Space Issues (optional section)	276
18.10 Bandwidth: How to Read the <i>San Francisco Chronicle</i> Business	
Page (optional section)	277
19 Machine Learning	281
19.1 Introduction to Machine Learning	281
19.2 Parallel Machine Learning	281
19.2.1 Supervised Learning	281
19.2.2 Unsupervised Learning	281
20 Introduction to Parallel Matrix Operations	283
20.1 “We’re Not in Physicsland Anymore, Toto”	283
20.2 Partitioned Matrices	283
20.3 Parallel Matrix Multiplication	285
20.3.1 Message-Passing Case	285
20.3.2 Shared-Memory Case	287
20.3.3 R Snow	292
20.3.4 R Interfaces to GPUs	292
20.4 Finding Powers of Matrices	292

20.4.1 Example: Graph Connectedness	292
20.4.2 Example: Fibonacci Numbers	294
20.4.3 Example: Matrix Inversion	295
20.4.4 Parallel Computation	295
20.5 Solving Systems of Linear Equations	296
20.5.1 Gaussian Elimination	296
20.5.2 Example: Gaussian Elimination in CUDA	297
20.5.3 The Jacobi Algorithm	299
20.5.4 Example: OpenMP Implementation of the Jacobi Algorithm	299
20.5.5 Example: R/gputools Implementation of Jacobi	301
20.6 Eigenvalues and Eigenvectors	301
20.6.1 The Power Method	301
20.6.2 Parallel Computation	303
20.7 Sparse Matrices	303
20.8 Libraries	304
21 Recurring Performance Issues	305
21.1 Communication Bottlenecks	305
21.2 Load Balancing	306
21.3 “Embarrassingly Parallel” Applications	306
21.3.1 What People Mean by “Embarrassingly Parallel”	306
21.3.2 Iterative Algorithms	307
21.4 Static (But Possibly Random) Task Assignment Typically Better Than Dynamic	308
21.4.1 Example: Matrix-Vector Multiply	308
21.4.2 Load Balance, Revisited	310
21.4.3 Example: Mutual Web Outlinks	311
21.4.4 Work Stealing	312
21.4.5 Timing Example	312
21.4.6 Example: Extracting a Submatrix	313
21.5 Latency and Bandwidth	314
21.6 Relative Merits: Performance of Shared-Memory Vs. Message-Passing	315
21.7 Memory Allocation Issues	315
21.8 Issues Particular to Shared-Memory Systems	316
Index	317

Preface

The goal of this book is to provide an introductory level overview of modern scientific computing. This book is intended to prepare students to be able to implement algorithmic techniques for high performance computing for applications. All the programs are developed in C programming language, which is the most widely used language for scientific computing.

The book consists of several parts that are developed to be independent of other parts. The book begins with an introduction to parallel computing. and then introducing the tools useful for developing the codes. This is followed by a crash course on C programming. It is assumed that the students have basic familiarity with the programming logic. Example codes are also provided.

Next, OpenMP is introduced. OpenMP provides a model for shared memory machines. OpenMP is the entry point to get started with parallel programming. It provides an easy to use platform for parallelization which is supported by C/C++ and Fortran on a wide variety of architectures. This part covers most of the major features of OpenMP, for example, its various constructs and directives for specifying parallel regions, work sharing, synchronization and data environment.

The Message Passing Interface Standard (MPI) is presented next. The goal of this part is to introduce the Message Passing Interface for writing message passing programs. MPI is the gold standard for writing message passing programs on HPC platforms. This part teaches students how to develop and run parallel MPI programs. This part begins with basic concepts for getting started with MPI, followed by a detailed look at the MPI routines that are widely used. A variety of examples are provided.

The next part addresses the application of GPU's in HPC. The fundamental tools and techniques for running C codes on GPUs with CUDA are introduced. CUDA parallelization, memory optimization, and migration between the CPU and GPU are shown. Students will encounter techniques that can be considered general parallel programming paradigms and will utilize CUDA streams for additional concurrency.

The selection of numerical methods was influenced by the practice. The finite difference method is commonly used for non-curved boundaries; here only a bare-bone discussion of it is introduced in the context of the solution of the elliptic partial differential equations. Simplest form of the Fast Fourier Transform and its parallelization is discussed. The goal of the Machine Learning part is to present the key concepts that form the core of machine learning. Machine learning has become an integral part of many modern applications. This part provides a view

to understand how to utilize algorithms for parallel Machine Learning.

INTRODUCTION TO PARALLEL COMPUTING

Parallel hardware has seen an increasing advances in recent decades. Nowadays, personal laptops and desktop use multicore processors. Computing clusters are essential for simulation in Science and Engineering and familiarity with parallel programming and architecture is a must for computer science majors. Real World problems are complex and without parallel computing simulations that are too big are too expensive to carry out. Massively large scale computations, particularly in Climate Modeling, Drug Discovery, and Data Analysis, rely on computer clusters comprising of thousands of processors. From this point view, it is important to understand the fundamentals, tools, and algorithms for parallel programming for currently available platforms. In what follows, an overview of parallel programming models are introduced with an emphasis on design of parallel programs illustrated with examples.

1.1 Serial Computing

Traditionally, codes were written for serial computation, i.e. running a single job at a time. In such case, a problem is divided into a series of instructions, which are consequently executed sequentially on a single processor. In this manner, instructions are execute one at a time. To understand this process further, it is best to first understand the structure of a sequential computer, which consists of main *memory*, a *central processing unit* (CPU), also called processor or core, that are connected via an *interconnection*. A memory consists of a collection of storage locations; both instructions and data are stored in the memory. A memory address points to the location of the memory where the contents, instructions or data, can be accessed. CPU consists of two units: Control Unit and Arithmetic Logic Unit. The Control Unit is for controlling the executing of instructions and

the Arithmetic and Logic Unit execute the instructions. Registers store the data and information about the execution in the CPU. This structure layout is known as a von Neumann machine and this separation of CPU and memory, which stores both program and data, is called the von Neumann bottleneck. This is because in the process of fetching the data and executing the instructions, instructions and data are transferred between the CPU and memory via the interconnect, and therefore the rate at which data can be transferred is limited by the interconnect. As an example, consider the following C code:

```
void add_store (double *a, double *b, double *c) {  
    *c = *a + *b;  
}
```

Here what happens:

- load **a** and **b** from memory to register
- load, combined with an arithmetic addition, to another register
- store back the sum result to memory location of **c**,

where compilers manage memory and registers. The cost of executing this routine includes the cost of the arithmetic operation and the memory accesses, load/store. Data transfers (load/store) have two costs:

- Latency: cost to load or store 1 word
- Bandwidth: average cost per byte to load/store a large chunk.

The performance of an interconnect depends on the latency and the bandwidth. The large chunk is typically more than one floating point number and therefore to increase performance many memory requests must be issued. However, due to latency, it will take time for the data to be transferred. To reach full bandwidth speed, a volume of independent data equal to the bandwidth times the latency has to be in flight, also known as the Little's law:

Concurrency = Bandwidth * Latency.

The overall performance is therefore determined not just by the processor performance, but also the speed of the memory system to transfer data. The clock speed of processors have increased at over years, while memory access times lag behind. In modern computer systems, this bottleneck is addressed by having a hierarchy of successively faster memory devices called caches. Cache is fast, but expensive, memory that takes advantage of the locality of the data. Many programs have a high degree of data locality; for example, when looping over an array, accessing

things nearby previous accesses, called spatial locality, or reusing an item that was previously accessed, called temporal locality. Accessing instructions or data item already residing in cache is cheap, called cache hit. If those not in cache, which is called a cache miss, then there will be a need to access next, slower level of memory. Computer hardware directly manages the cache memory. On-chip caches are faster, but limited in their sizes. Approaches for handling memory access can be viewed as:

- Eliminate memory operations by saving values in small, fast memory (cache or registers) and reusing them.
- Take advantage of better bandwidth by getting a chunk of memory into cache (or vector registers) and using whole chunk.
- Take advantage of better bandwidth by allowing processor to issue multiple loads/stores with a single instruction, or in parallel with other loads/stores/-operations.

Most programs run below maximum performance, mainly due to interconnect. Consider an example of computing inner product, involving one multiplication and addition per cycle. A processor operating at 1 GHz connected to a Dynamic Random-Access Memory (DRAM) with a latency of 100 ns with no caches. Considering two multiply-add units and executing 4 instructions per cycle, the peak performance is 4 GFLOPS. 100 ns latency means that at each cycle, processor waits 100 cycles before executing the next cycle, since each cycle is performed at 1 ns. On this platform, an inner product computation can be done with a speed of 10 MFLOP, which is much smaller than the peak performance. Next we review parallel computing to highlight the need for improving the performance loss on a single processor.

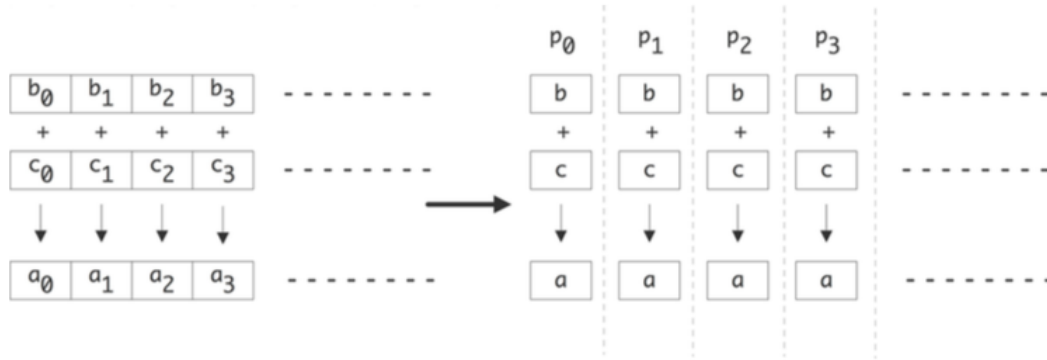
1.2 Parallel Computing Overview

In the previous section, we reviewed the concept of instructions can be executed simultaneously “in flight”, known as instruction-level parallelism, where users have no control over it. On the other hand, users can explicitly schedule more than one instruction to be handled by multiple processors. In the simplest sense, a problem is broken into multiple parts, and consequently further broken down to a series of instructions that can be executed concurrently on different processors. The control and coordination mechanism is employed by the programmer. In the following, we will review the hardware, the programming, and the concepts of this type of parallelism.

Consider a very simple example of adding two vectors:

```
for (i = 0; i < N; i++)
    a[i] = b[i] + c[i]
```

to be executed on N processors. Ideally, N processors can execute the single instruction $a = b + c$, as depicted below.



Ideally, this parallel algorithm should execute in one unit of time that takes to do one operation and in general, the execution time is linearly proportional to the number of processors. It is however not always simple to parallelize the problem. Often the algorithms must be redesigned to accommodate for parallelization. Even after that, the algorithm may not linearly reduce the execution with the number of processors.

There are a number of ways to classify computer structures:

- Single instruction stream, single data stream, or SISD, where only one instruction stream is being acted on by the processor and only one data stream is loaded at any one cycle.
- Single instruction, multiple data, or SIMD, where all processing units execute the same instruction at any given clock cycle and each processing unit can operate on a different data element.
- Multiple instruction, multiple data (MIMD), where every processor may be executing a different instruction stream and working with a different data stream.

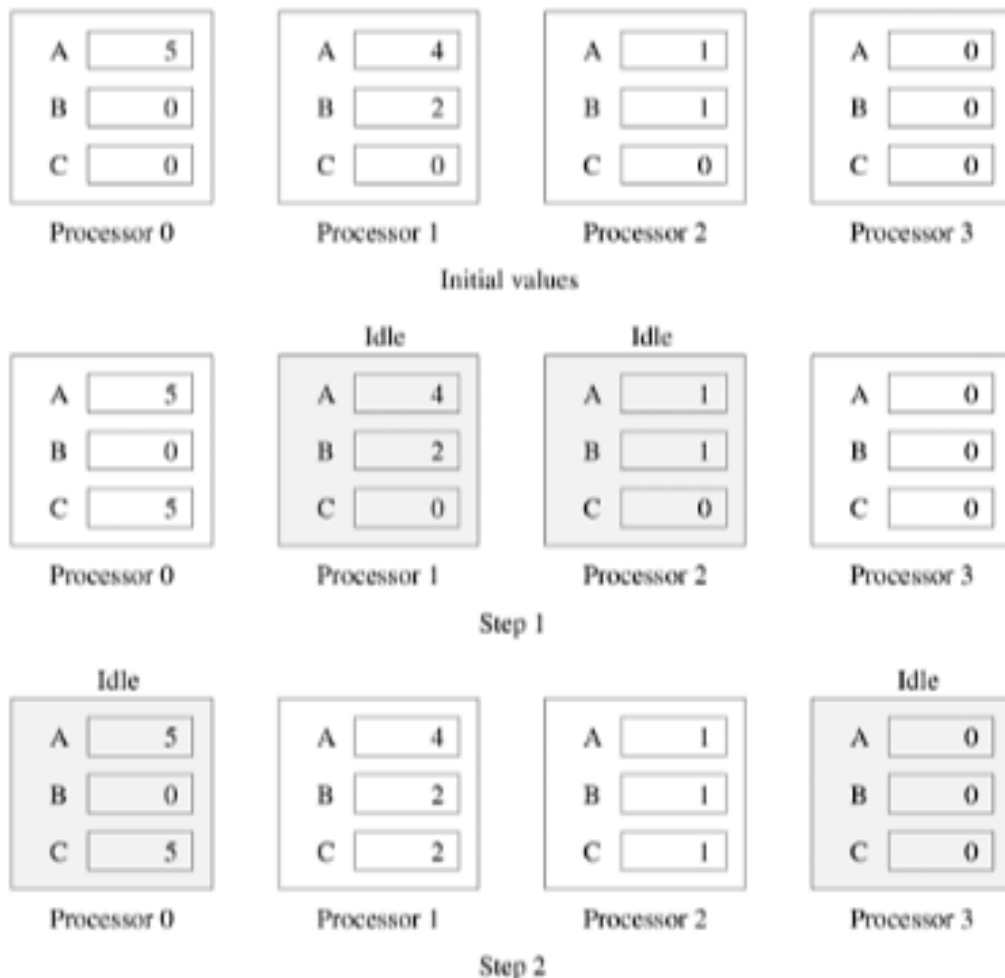
An example of parallelism for SIMD architectures is the vector addition mentioned above, which can be modified as

```
for (i = my_low; i < my_high; i++)
    a[i] = b[i] + c[i]
```

where each processor executes the same addition operation concurrently and syn-

chronously. SIMD architecture relies on the highly structured (regular) nature of the underlying computations, such as the vector addition above, and therefore ideal for parallelizing simple loops operating on huge arrays of data. The example above is called data-parallelism, where data is divided among the processors that each apply the same instructions to their data subsets. While SIMD computers are very efficient on large data parallel problems, they are not suitable for some parallelizations; for example, consider a conditional execution

```
if (B == 0)
    C = A;
else
    C = A/B;
```

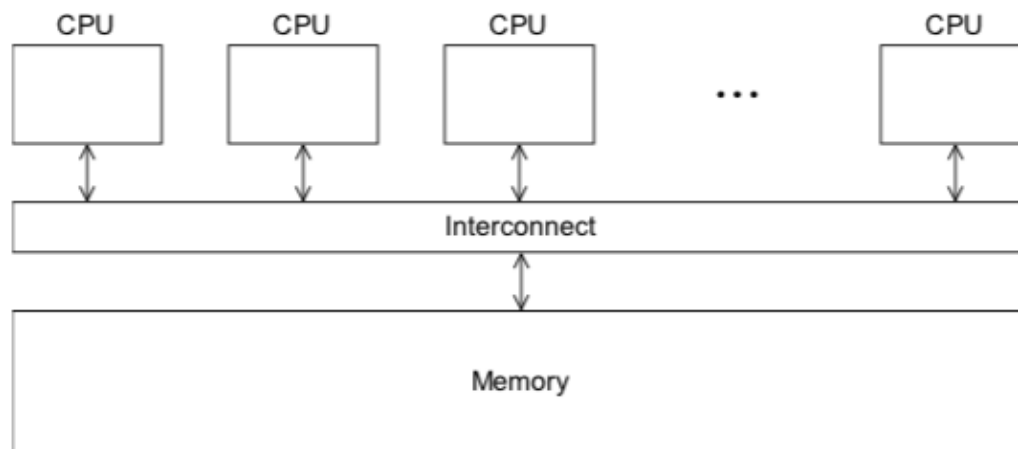


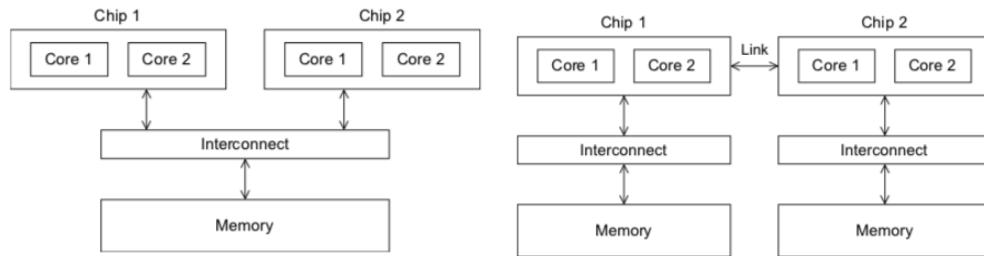
As shown, some processors become idle, leading to poor utilization of resources. Such drawbacks have made SIMD architecture less desirable and nowadays, the

most common parallel architecture is MIMD. We will mainly focus on MIMD machines that operate in Single Program Multiple Data (SPMD) mode, where the same executable act on their copy of the same program simultaneously that will not in general be synchronous as in SIMD machines. The lack of synchronization can be due to processors working on different amounts of data, known as load unbalance, leading to the loss of performance. There are two main types of SPMD models: shared-memory and distributed-memory models.

1.2.1 Shared Memory Model

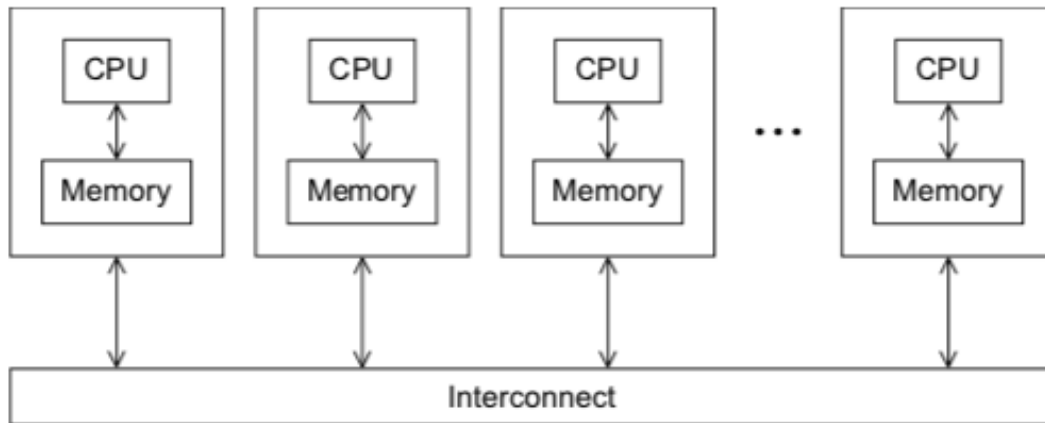
A shared-memory model is a parallel platform of processors that operate autonomously connected to a memory by an interconnection, and each processor has access to all memory locations. If the access time is the same for each processor, the system is called Uniform Memory Access (UMA), most commonly represented Symmetric Multi Processing (SMP) programming model. On the other hand, if the access time is different for certain processes, the system is called Non-Uniform Memory Access (NUMA). NUMA machines are often built by linking SMP machines, where one SMP can directly access memory of another one, however, memory location to which a processor is directly connected can be accessed more quickly than a memory location that must be accessed through the link. If and update to a location in shared memory done by one processor can be visible to all the other processors, then the system is called cache coherent. Below are diagrams of shared memory, UMA, and NUMA, systems:





1.2.2 Distributed Memory Model

Parallel programming is relatively straightforward if all processors have access to any memory location. Moreover, it provides data sharing between processors that is fast and uniform. However, the main disadvantage is the lack of scalability between memory and processors, due to increased traffic on the memory-processor interconnect, and for cache coherent systems, due to the cache-memory management. One solution is to have the processors have their own local memory and therefore each processor operates independently and any updates to its local memory is not visible to other processors. With this setup, the only way one processor can exchange information with another is through a communication network that connects inter-processor memory. This communication is set up by the programmer, as well as synchronization between tasks. Below is a diagram of a distributed memory system. The most widely used distributed memory machines are computer clusters and supercomputers. The interconnection network typically is Ethernet. Often, each computational unit, is a shared memory systems with multiprocessors, linked to other units through communication network. These are called hybrid systems. The individual units can be built from different types of hardware, making the system heterogeneous. Below is diagrams of a distributed memory systems:



1.3 Case study: Matrix Multiplication

Use of performance models to understand performance • Attainable lower bounds on communication • Simple cache model • Warm-up: Matrix-vector multiplication • Naive vs optimized Matrix-Matrix Multiply • Minimizing data movement • Beating $O(n^3)$ operations • Practical optimizations (continued next time)

PROGRAMMING TOOLS

Programming tools are needed to assist a programmer with basic aspects of programming. Programming environment consists of programming tools to facilitate programming and to enhance programming ease and productivity. Programming tools are needed for writing the actual code, editing existing code, to compile, debug and linking the code. In what follows, we review two distinct types of programming environments: Command-line and Integrated Development Environments.

2.1 Command-line Environments

Command-line environment provides a collection of commands that can be used to edit codes, compile source codes, debug, and run programs. However, due to its nature, the beginners must to put in a some efforts to master the practice. The command-line environment can makes the use of scripting and batch files tremendously easy. The commonest example of this is the UNIX Shell. UNIX shells allows users to enter commands by providing a command-line interface. Typed in commands are translated by the shell into something the UNIX kernel can process. One of the widely used Shells available is `bash`.

Despite the fact that most modern tools have a graphical user interface (GUI), command-line user interfaces (CLUIs) still have their place; it is easier to see and understand what the computer is doing on your behalf and more importantly, it is easier to combine CLUI tools than GUI tools. In what follows, some basics of shell will be introduced. The command shell manages a user's interactions with the operating system by:

- Reading commands from the keyboard
- Figuring out what programs the user wants to run

- Running those programs
- Displaying their output on the screen
- Looks (and works) like an interactive terminal:

```

shahriar@earth: ~/Teaching/Courses/GraduateSummer/Shell/None
File Edit View Search Terminal Help
shahriar@earth:~/Teaching/Courses/GraduateSummer/Shell$ ls
Afkhami  None
shahriar@earth:~/Teaching/Courses/GraduateSummer/Shell$ cd None/
shahriar@earth:~/Teaching/Courses/GraduateSummer/Shell/None$ ls
Makefile
shahriar@earth:~/Teaching/Courses/GraduateSummer/Shell/None$ ls -alt
total 8
drwxrwxr-x 4 shahriar shahriar 4096 Jun 18 16:24 ..
drwxrwxr-x 2 shahriar shahriar 4096 Jun 18 16:24 .
-rw-rw-r-- 1 shahriar shahriar    0 Jun 18 16:24 Makefile
shahriar@earth:~/Teaching/Courses/GraduateSummer/Shell/None$

```

In bash:

- Output can be redirected using `>` and `<`.
- Commands can be combined using pipelines.
- The history command can be used to view and repeat previous operations, while tab completion can be used to save re-typing.
- Directories (or folders) are nested to organize information hierarchically.
- Use `grep` to find things in files, and `find` to find files themselves.
- Programs can be paused, run in the background, or run on remote machines.
- The shell has variables like any other program, and these can be used to control how it behaves.

In UNIX, all data is organized into files, that are in turn organized into directories. A tree-like structure, filesystem, is used for file management. The File System:

- The file system is the set of files and directories the computer can access
- Everything in a particular directory must have a unique name
- On Unix, the file system has a unique root directory called /
- Every other directory is a child of it, or a child of a child, etc.
- A path is a description of how to find something in a file system
- An absolute path describes a location from the root directory down
- A relative path describes how to find something from some other location

Here we also list basic commands (most shell commands have cryptic names):

pwd	print working directory
ls	listing
cd	changing working directory
.	current directory
..	parent directory: directory immediately above this one
mkdir	creating directory
cp	copy a file
nano	text editor
rm	delete a file
rmdir	delete a directory
mv	move or rename a file or directory
wc	word count
cat	catenate file

By convention, flags start with “-”, as in “-c” or “-l”: “ls -F”, “ls -a”, “ls -Fa”, and “ls -s”. The real strength of the shell is its ability to combine simple utilities in useful ways called pipes and filters:

>	redirect the output to a file
<	redirect the input to a file
	pipe/connect processes together
*	pattern-matching wildcard
?	matches any single character

Some useful commands:

- awk - Text processing language: Find and Replace text within file(s)

- `chmod` - Change access permissions
- `grep` - Search file(s) for lines that match a given pattern
- `history` - Command History
- `locate` - Find files - simple but fast
- `whereis` - Search the user's \$path, man pages and source files for a program
- `xargs` - Execute utility, passing constructed argument list(s)

Linux Process Control:

<code>top</code>	display Linux tasks
<code>kill</code>	terminate a process
<code>ps</code>	report a snapshot of the current processes
<code>ctl C</code>	kill a running job
<code>&</code>	run in the background
<code>jobs</code>	tell processes are currently running in the background
<code>fg</code>	brings the background job into the foreground
<code>ctl Z</code>	pause the job and return control to the shell (use <code>fg</code> or <code>bg</code> to resume)

Environment variables:

<code>HOME</code>	The current user's home directory
<code>HOSTNAME</code>	This computer's name
<code>HOSTTYPE</code>	What kind of computer this is
<code>PATH</code>	Where to look for programs
<code>PWD</code>	Present working directory
<code>SHELL</code>	What shell is being run
<code>TEMP</code>	Where to store temporary files
<code>USER</code>	The current user's ID

If we want to set some variables' values automatically every time we run a shell, we can put the command to do this in a file called `.bashrc` in our home directory. The `."` character at the front prevents `ls` from listing this file unless we specifically ask it to using `-a` — we normally don't want to worry about it — and the `"rc"` at the end is an abbreviation for "run control".

2.2 Integrated Development Environments (IDE) and Text Editors

An Integrated Development Environments (IDE) is a collection of applications used for source-code editing, compilation tools, a debugger, and a version control system, and sometimes a graphical user interface (GUI), and an object-oriented software development support. An IDE includes a text editor, sometimes more than a simple one with convenient features such as syntax highlighting, interfaces, and navigation tools. As a centralized environment, it also includes a version control systems, compiling and build tools, and debugging tools. Using an IDE improves workflow and allows an easier detection of errors. Overall using an IDE improves the efficiency code development. For applications in this book, multi-language IDEs are most suitable; the support for multiple programming languages and best suited for beginner programmers. Examples of this type of IDE is the Visual Studio IDE and Eclipse, which are known for their useful features, such as language plugin.

Text editors are used to edit an existing file or to create a new file from scratch to just read a text file. There are many text editors in Unix, including screen-oriented text editors such as `vi`, `nano`, and `vim`, to others with more advanced features. Modern text editors are easy to use and navigate, have find and replace, syntax highlighting, and are customizable. Sublime Text editor is one of the best code editor for a complete development environment. Emacs is the best code editor with a highly customizable environment. Emacs also integrates with GNU debugger (GDB) to provide an IDE interface, also supporting compiling, running, and testing programs, as well as shell commands.

2.3 Compilers and makefiles

There are two ways to translate a program into computer instructions i) **interpreting** or **compiling**. An interpreter reads a high-level program and translates and carry out commands line-by-line. A compiler is a program that translates a high-level program all at once, creating an executable file that can be run later. The high-level program is called **source code** (".c" and ".h" files for C codes), the translated program is called the **object code** (".o"), and the executed program is called the **executable**. A compiler reads the source code, translates it, builds an object file, and creates a file that is the executable. Consider the following "Hello, World." example:

hello.h:

```
void print_hello ();
```

hello.c:

```
#include <stdio.h>
#include "hello.h"

void print_hello () {
    printf("Hello, world.");
}
```

main.c:

```
int main () {
    hello ();
    return 0;
}
```

This program can be compiled using the GNU C compiler as follows:

```
$ gcc main.c hello.c
$ gcc -o hello.exe main.c hello.c
$ gcc -c main.c hello.c -o hello.o main.o hello.o
```

where the first line compiles and links into an executable called “a.out”, the second line is the same as the first line with an executable name “hello.exe”, and the third line translates the C code into object files and then the linker links the object files and creates the executable. The advantage of the latter is that if only one of the source codes is modified, only the updated file will be recompiled. This avoids unnecessarily compiling all the files which in the case of large projects can be inefficient. To link external libraries, for example, one can add the linking line:

```
$ gcc -c main.c hello.c -o hello.o main.o hello.o -lm
```

where `-l` option links the standard math library, `m`. Compilers accept many options, such as optimization options, the “`-O`” options.

Big programs have thousands of lines of code, organized into hundreds of source code files, with their corresponding header files. The make utility is a tool for managing and maintaining large codes. When only some of the source files are modified, but not all,

t make can automatically determine what needs to be recompiled. A file called “Makefile” (or “makefile”) which contains the dependencies to build a target, has a set of rules to determine which parts of a code needs recompilation. In a sense, Makefile is a way to automating repeated procedure (or other complex tasks with dependencies). A Makefile consists of dependency rules, macros and suffix rules. Here is a Makefile example of the “Hello, world.” program above:

```
# This is a comment line.
# CC is a macro for the compiler.
CC = gcc
# CFLAGS will be the options passed to the compiler.
CFLAGS = -c -Wall

all: exe

exe: main.o hello.o
    $(CC) main.o hello.o -o exe

main.o: main.c
    $(CC) $(CFLAGS) main.c

hello.o: hello.c
    $(CC) $(CFLAGS) hello.c

clean:
    rm -rf *.o
```

The make utility looks at any dependency file and if it has a modification time more recent than its target file, it then recreates the target. Let’s look at the Makefile above:

```
exe: main.o hello.o
<tab> $(CC) main.o hello.o -o exe
```

- A rule consists of a target, here `exe`, dependencies, here `main.o hello.o` and commands to build the target: `$(CC) main.o hello.o -o exe`.
- `<tab>` character cannot be replaced by spaces.
- A “target” is the name of a file (here the executable). or an action (for example, `clean`, here).
- “dependency” files are used as to build the target.

- Commands in a rule are shell commands to be executed.
- Typing `make` execute the first target in Makefile, here “exe”.
- “clear” here is a phony target; i.e. no dependencies, and just a list of commands.
- Macros allow to avoid repeating entries; for example, here: `CC = gcc`.

References: <https://www.gnu.org/software/make/>

2.4 Debugging

As codes become longer and longer, it becomes more and more complex, and therefore easier to introduce errors. Programming errors are called **bugs** and the process of finding the cause of them and correcting them is called **debugging**. Compiler can detect syntax error; a program cannot be compiled is syntactically incorrect. The error that does not show up until the program is run is called run-time error. Logical and semantic errors do not cause compilation error and run successfully, but it will produce the right outcome.

Debugging is the integral part of programming. The GNU debugger, GDB, can be used to debug programs compiled with the the GNU C compiler, gcc. The compilation must be done with the compiler option “-g”; e.g.

```
$ gcc -g -c -o hello.exe main.c hello.c
```

To start GDB, type:

```
$ gdb hello.exe
```

which will bring up a debugging session. There are a number of gdb commands that are useful for a debugging session:

r(un) [arglist]	Runs the program in GDB with optional argument list.
b(reak) [file:]function/line	Puts a breakpoint to stop the program when reached.
delete [file:]function/line	Removes the breakpoint at the given point.
c(ontinue)	Resumes execution of the program after it is stopped.
n(ext)	When stopped, runs the next line of code, stepping over functions.
s(tep)	When stopped, runs the next line of code, stepping into functions.
q(uit)	Exits GDB.
p(rint) expr	Prints out the given expression.
display var	Displays the given variable at every step of execution.
l(ist)	Lists source code.
help [command]	Gives help with a specified command.
bt	Gives a backtrace

2.5 Version control

Version control is a great tool for software development. It synchronizes files, keeps track of changes, undoes changes (think of unlimited undoing), and keeps the master copy of the file, amongst many other features. It also allows multiple users to work on the same project in parallel. All large scale coding projects use version control; it allows more robust software development and a more coordinated and organized teamwork and collaboration. Version control saves the data or files after making modifications, allowing reverting back and retrieve specific versions. Consistent version control allows programmers to test new algorithms without fearing breaking versions that work. The way it works is that it provides a trace of changes in case the programmer needs to access earlier revisions. A version control system provides:

- reverting to past versions
- comparing different versions
- pushing full version history to a repository
- pulling history back from the repository
- merging versions
- allowing multiple developers to work together:
 - to combine versions
 - to track who made changes
 - to work in parallel

– to share unfinished work

There are a number of automated version control systems, such as CVS or Subversion. We recommend here a modern system, Git, that do not need a centralized server to host the repository (the repository is the folder inside our project folder that keeps track of all the changes made to the files in our project and record that history over time). Merging tools on Git also make it easy for multiple developers to work on the same files simultaneously. Below we will review Git on the command line.

When using Git for the first time, things need to be configured. Below are a few Git commands for configurations (on a command line, Git commands are written as `git verb`, where verb is what we actually want to do):

- `git config -global user.name "your name"`
- `git config -global user.email "your e-mail"`
- `git config -global core.autocrlf input` (For macOS and Linux)
- `git config -global core.editor "emacs"`
- `git config -global core.editor "Vim"`
- `git config -list`
- `git -help`

Once Git is configured, we can start using it by creating a repository first. Also note that the created repository automatically includes subdirectories (and their files). When for example we add a file to the repository, Git will indicate that a new file is created. Before telling Git to track the file, using `git add "filename"`, the

`git status` command will show "untracked files" message. Finally, to have Git record the changes, a "commit" must be run, using `git commit` command.

git init	Make the working directory a repository
git status	Show the status of the project
rm -rf .git	Remove the entire Git history of a project
git add <i>file</i>	Add file contents to the index
git commit	Record changes to the repository
git pull	Fetch from a remote repo and try to merge into the current branch
git push	Push new branches and data to a remote repository
git log	Show commit logs
git diff	Show changes between commits
git diff HEAD	Show the difference between the last commit
git diff HEAD~1	Show the difference between one before HEAD
git checkout	Check out a branch or paths to the working tree
git branch	List, create, or delete branches
gitk	Show GUI history
git clone url [dir]	Clone a remote repo to dir

For example, consider the previous example, `main.c`, `hello.c`, `hello.h` and the `Makefile`. To create a new `git` repository, type:

```
$ git init /PATH/TO/CODE/DIRECTORY
```

which will create the repository including the file `.git`. That directory should only be accessed through `git`. Next add the files to the repository, assuming the working directory is `/PATH/TO/CODE/DIRECTORY`:

```
$ git add main.c hello.c hello.h Makefile
```

Typing

```
$ git status
```

will show something like:

```
Shahriars-MacBook-Pro-2:git shahriar$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)

        new file:   Makefile
        new file:   hello.c
        new file:   hello.h
        new file:   main.c

Shahriars-MacBook-Pro-2:git shahriar$
```

Typing:

```
$ git commit -m "commit message" main.c hello.c hello.h Makefile
```

will make these files part of the repository, with proper information included as the message. Now suppose that `main.c` is modified and updated. Then `git status` will show that the changes aren't committed yet. However, if it turns out that the changes will break the code, you can return to the previous version by typing:

```
$ git checkout -- main.c
```

C PROGRAMMING

3.1 The first program

The classic “Hello World!” in C looks like this:

```
1  /*
2   Hello World
3
4   Demonstration showing a simple program that is self-contained.
5
6   Program written by David Chopp
7
8   Editing History:
9
10  2/14/14: Initial draft
11  7/29/17: Changed formatting of comments
12 */
13
14 // Programs often require functions from built-in libraries ,
15 //      or user-written files
16 // Interface information for those functions are stored in header
17 //      files
18 //      and loaded before the code is compiled.
19 // This program uses terminal I/O, so it uses the stdio library.
20 //      This is very common...
21 #include <stdio.h>
22
23 /*
24  int main(int argc, char* argv[])
25
26  The main program is the starting point for a C program.
27  There can only be one "main" function. All others must be
28  uniquely named something else.
29
```

```

30 Inputs:
31 int argc: When the program is run, argc contains the number of
32 arguments given to the program, e.g. if you type
33 "hello there" to the command line, then argc will have
34 the value 2.
35
36 char* argv[]: When the program is run, argv contains an array of
37 strings, where each string is what was typed, e.g. if
38 you type "hello there" to the command line, then
39 argv[0] will have the value "hello" and argv[1] will
40 have the value "there"
41
42 Outputs:
43 int: The main program should return 0 if successful, and return
44 a non-zero value if it encountered an error.
45 */
46
47 int main(int argc, char* argv[])
48 {
49     // It is important to get in the habit of thoroughly commenting
50     // your code.
51
52     printf("Hello World!\n");
53
54     // All done, no errors, so return 0
55
56     return 0;
57 }

```

Here we review several features of this code. Lines 1-12 are comments and any line beginning with `//` also indicates that it is a **comment**. A comment is used towards readability and documentation. Compiler ignores everything that is commented. Places to add comments are the top of each file, to include the information about the contents of the file, at the top of each function, to describe the function, the arguments, and the output of the function, and descriptive information about the logical steps of the program.

In line 47, `main` is a special name that indicates the place in the program where execution starts. At the beginning, the program executes the first statement in `main` and it continues, in order, until it gets to the last statement, and then it exits. All statements should end with a semicolon (`;`).

C uses squiggly-braces (`{` and `}`) to group things together. In this case, the `printf` statement is enclosed in squiggly-braces, indicating the *scope* of `main`. Also, notice that the statement is indented, which helps to show visually which lines are inside the definition. We showed earlier how to compile and run this program. The phrases that appear in quotation marks are called **strings** which can contain any

combination of letters, numbers, punctuation marks, and other special characters. The the backslash in `printf \` is to add a new line to the output.

Unlike most languages, C has no input or output statements. Instead, a set of I/O functions is provided as a part of the standard library. One such function is `printf`. Prior to calling a standard library function, we should inform to compiler where to find the function. In the case of standard I/O, including calling `printf`, we do this by using `#include <stdio.h>`. This is called a directive for the C *preprocessor*, which processes the source code before compilation. In this case, the header file is `stdio.h`. The header file must be placed at the top of the source file, prior to any function definition. We can think of the header file being replaced by its source file that

3.2 Variables

C is a powerful language in terms of its ability to manipulate **variables**. There are different types of variables. When variables are declared in C, their types need to be determined. Common arithmetic data types are:

Type name	Size (byte)	Description
<code>char</code>	1	Single Character
<code>bool</code>	1	Boolean
<code>int</code>	4	Integer
<code>float</code>	4	Single precision floating point
<code>double</code>	8	Double precision float
<code>short int</code>	2	Short integer
<code>long int</code>	8	long integer
<code>long double</code>	16	Quadruple precision floating point
<code>unsigned char</code>	1	Unsigned single character
<code>unsigned int</code>	4	Unsigned integer
<code>unsigned short int</code>	2	Unsigned short integer
<code>unsigned long int</code>	8	Unsigned long integer

While the sizes listed above are typical sizes, we must use `sizeof` function to determine the exact size values on a particular system. A declaration of a variable looks like this:

```
char foo;
```

to create a `char` variable. Obviously, the range and precision of these types are important to a programmer. For example:

Type name	Limits Macros	Range
<code>char</code>	<code>[CHAR_MIN, CHAR_MAX]</code>	<code>[-128, 127]</code>
<code>int</code>	<code>[INT_MIN, INT_MAX]</code>	<code>[-2147483648, 2147483647]</code>
<code>float</code>	<code>[FLT_MIN, FLT_MAX]</code>	<code>[1.175494 × 10⁻³⁸, 3.402823 × 10³⁸]</code>
<code>double</code>	<code>[DBL_MIN, DBL_MAX]</code>	<code>[2.225074 × 10⁻³⁰⁸, 1.797693 × 10³⁰⁸]</code>

Macros are character strings that replace their defined value at the compilation time. For example, in `limits.h`, `INT_MAX` is defined as

```
#define INT_MAX 2147483647;
```

Another important value that a programmer needs to know is the machine epsilon, an upper bound on the relative approximation error due to rounding in floating point arithmetic. For example, for a `float` type, the machine precision is 1.192093×10^{-7} , meaning that the single precision float can accurately represent a number of 7 significant digits. There is a speed trade-off in choosing a greater precision. Higher precision arithmetic is slower. The macro for machine epsilon is defined, for example for `float`, as `FLT_EPSILON`.

Variables are assigned values, either when they are declared or later, before being used. This is done with an **assignment statement**.

```
int index;
long double x = 11.111;
index = 0;
```

We should also note that uninitialized variable are not set to zero by default. Also, for example, a string cannot be stored in an `int` variable; in general, variables and values have the same type.

Generally, any variable names can be used; however, certain keywords are reserved in C because they are used by the compiler to parse the structure of the program. For example, name of a variable cannot be `int`, `char`, `void`, and so on. The complete list of keywords can be found at <http://www.ansi.org/>.

3.3 Inputting and Outputting

The input and output are mainly used in two ways: outputting diagnostic information, storing data. We will review a program that reads and writes files, parses input, as well as terminal I/O.

We can output the value of a variable using `printf` command:

```
int hour = 11, minute = 59;
```

```
char colon = ':';
```

```
printf ("The current time is: \n");  
printf ("%d%s%d\n", hour, colon, minute);
```

This program creates two integer variables named `hour` and `minute`, and a character variable named `colon`. It assigns appropriate values to each of the variables and then uses `printf` to output statements to generate the following:

```
The current time is:  
11:59
```

“\n” (called a escape sequence) represents a new line. The variables `hour` and `minute` are of `int` type and `%d` tells `printf` that it corresponding argument has an integer type, when displaying its value. In general, `printf` appears as:

```
int printf(char *format, argument1, argument2, ... )
```

`scanf()` is similar to `printf`, but it reads input and appears as:

```
int scanf(char *format, addr of argument1, addr of argument2, ... )
```

The first argument will be a format string, which specifies the expected form of the input. Below is the list of format specifiers, for both `scanf` and `printf` :

Format specifier	Description	Supported data types
<code>%c</code>	Character	<code>char</code>
<code>%d</code>	Signed integer	<code>int</code>
<code>%e</code>	Scientific notation of float values	<code>float</code> , <code>double</code>
<code>%f</code>	Floating point	<code>float</code>
<code>%g</code>	Similar as <code>%e</code>	<code>float</code> , <code>double</code>
<code>%hi</code>	Signed integer (short)	<code>short int</code>
<code>%hu</code>	Unsigned integer (short)	<code>unsigned short int</code>
<code>%ld</code>	Long integer	<code>long int</code>
<code>%lf</code>	Double precision floating point	<code>double</code>
<code>%Lf</code>	Quadruple precision floating point	<code>long double</code>
<code>%o</code>	Octal number	<code>int</code>
<code>%s</code>	String	<code>*char</code>
<code>%u</code>	Unsigned integer	<code>unsigned int</code>
<code>%x</code>	Hexadecimal of unsigned integer	<code>int</code>
<code>%n</code>	Prints nothing	
<code>%%</code>	Prints <code>%</code>	

The second argument in `scanf` must be the memory address, not the value, unlike `printf`. Reading and writing files are important when dealing with large amount

of data. Let's look at one example:

```
1 #include <stdio.h>
2 #include <math.h>
3
4 /*
5  int main(int argc, char* argv[])
6
7  The program opens a new file with the given name, writes some data
8  to
9  it, then reads those numbers back in and prints them to the screen.
10
11  Inputs: argc should be 2,
12  argv[1] will contain the filename to be created
13
14  Outputs: creates the file and writes some data to it.
15 */
16 int main(int argc, char* argv[])
17 {
18     // First thing is to create the file we will use for storage
19     // the "w" indicates we are opening the file for writing
20     FILE *fileid = fopen(argv[1], "w");
21
22     // Once the file is open, we can write to it with fprintf functions
23     // that act the same as the printf commands we learned earlier
24     // Note the use of M_PI. This is a useful macro that defines the
25     // number pi and is found in the header file math.h
26     fprintf( fileid, "%d %6.3f %s", 17, M_PI, "Hello World!\n");
27
28     // When we're done writing, we must close the file so that it is
29     // terminated properly
30     fclose(fileid);
31
32     // To read the data back in, we must define the variables
33     int num;
34     double pi;
35     char greeting[256];
36
37     // The file is closed, so let's open it again, but this time
38     // use "r" to indicate that we will read the file
39     // Note that we don't have to redeclare fileid because
40     // we did it already
41     fileid = fopen(argv[1], "r");
42
43     // Now we use fscanf the same way we learned scanf
44     // Note that greeting does not have an ampersand because greeting
45     // is
46     // an array, so greeting has type char* already
47     fscanf( fileid, "%d %lf %s", &num, &pi, greeting);
```



```

47 // All done reading, so close the file
48 fclose(fileid);
49
50 // Now print the results. Did we get what we expected?
51 printf("%d %lf %s\n", num, pi, greeting);
52 return 0;
53 }
54

```

The I/O is through the form of a `FILE*` type. Here we open the file with `fopen()`, it returns the `FILE*` (or `NULL` if failed) - “w” here means open a text stream for writing. All streams are automatically closed when the program exits, but for a good practice, it’s better to explicitly close any files opened for reading or writing. `fprintf` and `fscanf` are similar to `printf` and `scanf` with an additional argument to stream data through reading or writing from a file. There is a macro defined as `EOF`. This is what `fscanf` will return when the end of the file has been reached. Here is another example of reading formatted text from a file `colorcode.txt`:

```

black 1
blue 2
white 3
gray 4

#include <stdio.h>

int main(void)
{
    FILE *fp;
    char color[1024];
    int code;

    fp = fopen("colorcode.txt", "r");

    while (fscanf(fp, "%s %f %d", color, &code) != EOF)
        printf("%s color code is: %d\n", color, code);
    fclose(fp);
}

```

The `fscanf()` function skips leading whitespace when reading, and returns `EOF` on end-of-file or error.

Reading and writing in text format is extremely inefficient; binary formats offer advantages in that they are more efficient. With binary files, a raw stream of bytes are processed or reading and writing. Below is an example

```

#include <stdio.h>

int main(void)
{
    FILE *fp;
    unsigned char bytes[6] = {5, 37, 0, 88, 255, 12};

    fp = fopen("output.bin", "wb"); // wb mode for "write binary"!

    // fwrite arguments are:
    // * Pointer to data to write
    // * Size of each "item" of data
    // * Count of each "item" of data
    // * FILE*

    fwrite(bytes, sizeof(char), 6, fp);

    fclose(fp);

    fp = fopen("output.bin", "rb"); // rb for "read binary"!

    while (fread(&c, sizeof(char), 1, fp) != 0)
        printf("%d\n", c);
}

```

The arguments of `fwrite()` are the size of each item how many items to be read. `fread()` here reads back the data from the file, which is in binary format.

3.4 Operations

Basic arithmetic **Operators** are `+`, `-`, `*`, `/`, `(`, `)` with mathematical operations order of precedence. Recall also that processors have special hardware just for performing floating-point operations. Integer division may result in an unexpected output. When both of the operands of an integer division are integers, the result is also supposed to be an integer, always rounding down by definition, regardless. For example,

```

int x = 1;
int y = 2;
int z = x/y; \\ z will be assigned 0
int r = (x + 2)%y; \\ r will be assigned 1

```

A type cast can be specified, for example, as follows:

```
int x = 1;
int y = 2;
double w = x / y; \\ w will be assigned 0.0
double z = (double)x/y; \\ z will be assigned 0.5
```

Below is an example of using increment and decrement shortcuts:

```
1 #include <stdio.h>
2
3 int main(int argc, char* argv[])
4 {
5     // We'll use two integer variables: val and i
6     // val will show the result of the equation, and
7     // i will be the variable being incremented or decremented
8     int val = 0;
9     int i = 1;
10
11     // Before/after each operation, you can see the resulting values of
12     // the increment and decrement operators
13     printf("Initial values:\nval = %d, i = %d\n\n", val, i);
14     val = ++i;
15     printf("After val = ++i:\nval = %d, i = %d\n\n", val, i);
16     val = i++;
17     printf("After val = i++:\nval = %d, i = %d\n\n", val, i);
18     val = --i;
19     printf("After val = --i:\nval = %d, i = %d\n\n", val, i);
20     val = i--;
21     printf("After val = i--:\nval = %d, i = %d\n\n", val, i);
22
23
24     return 0;
25 }
```

The same mathematical operations that work on integers also work on characters:

```
char letter;
letter = 'a' + 1;
printf("%s", letter);
```

outputs the letter b.

3.5 Boolean Algebra

There is another type in C called **boolean**, taking only values of **true** and **false**. The condition inside an **if** statement or a **while** statement is a boolean expression.

Also, the result of a comparison operator is a boolean value. For example:

```
int x;
...

if (x) {
    \\ do something
}
else {\\ do something else
}
```

where `if (x)` is the same as `if (x != 0)`. The basic boolean operations include the operator `==` compares two integers and produces a boolean value and `!=` which produces true if the values of operands are not equal. The values `true` and `false` are keywords in C. For example,

```
while (true) {
    // loop forever
}
```

As mentioned earlier, boolean type is called **bool**.

```
bool YES;
YES = true;
bool NO = false;
```

The result of a comparison operator is a boolean, for example:

```
bool even = (n%2 == 0);    // true if n is even
bool positive = (x > 0);    // true if x is positive
```

and then use it as part of a conditional statement later

```
if (even) {
    printf("n is even\n");
}
```

The logical operators are: AND , OR and NOT, given by the operators: `&&`, `||` and `!`. For example:

- `x > 0 && x <= 10` true only if x is greater than zero AND less than or equal 10.
- `(x != 10)` is false if x is not equal 10.
- `x <= 0 || x > 10` true if either if `x ≤ 0` is true OR the `x > 10`.
- `max = x > y ? x : y` assigns the larger of two values x and y to max

Functions can also return `bool` values. For example:

```
bool isPositive (int x)
{
    if (x >= 0) {
        return true;
    } else {
        return false;
    }
}
```

or

```
bool isPositive (int x)
{
    return (x >= 0);
}
```

Finally, it is important to note to never trust equality (or inequality) operations when dealing with floating point values.

3.6 Mathematical Functions

There are mathematical functions available to use in C. These functions are declared in `math.h`, which provides a set of built-in functions that includes most of the mathematical operations. The math functions are invoked similarly to their mathematical notation:

```
double x = abs(x); \\ assigns |x| to x
double y = sin (x); \\the same as sin(x)
```

Other examples are `log(x)` ($\ln(x)$), `log10(x)` (takes logarithms base 10), and `log2(x)` (takes logarithms base 2). The values used with `sin` and the other trigonometric functions (`cos`, `tan`) are in *radians*. π to 15 digits can be calculate it using the `acos` function. The arccosine (or inverse cosine) of -1 is π :

```
double pi = acos(-1.0);
```

3.7 Functions

Functions are the backbone of C. Functions provide ease of understanding of the code and help speed the programming development, by providing reuseability.

Complex algorithms can be broken up into small modules that are easy to understand, debug, and test. Functions simplify algorithms and if developed properly, for example being as generic as possible, they can be used as building blocks, mainly for later code developments.

We have already used the `main` function, where the execution of the program starts, but the syntax for `main` is the same as for any other function definition. Other function examples are `printf` and `scanf`. Functions take arguments (or no argument) and return a value (or nothing). The arguments and return value types are predeclared. Function definitions look like:

```
double NAME ( LIST OF PARAMETERS ) {  
    STATEMENTS  
}
```

The list of parameters is the data, or no data, passed to the function. In this case, the function returns a double precision value. A function declaration is included in the header file and appears like:

```
double NAME ( LIST AND TYPE OF PARAMETERS );
```

Compiler needs to know about the function declaration before it is used. It is useful to plan properly based on whether the function is used only in one file, or being called in many other files. Here are two usage examples of the same function:

```
1 #include <stdio.h>  
2 #include <stdlib.h> // need this include for the atoi function  
3  
4 /* unsigned int squareOf(int n)  
5  
6     returns the square of the input integer  
7  
8     Input: number to be squared  
9  
10    Output: square of input value  
11 */  
12 // Note that squareOf is defined before it is called in main  
13 unsigned int squareOf(int n)  
14 {  
15     return n*n;  
16 }  
17  
18  
19 /*  
20 int main(int argc, char* argv[])  
21  
22     Function prints the square of the integer argument  
23
```

```

24 Inputs: argc should be 2
25 argv[1] will contain the integer input, can be positive or negative
26
27 Outputs: Prints the square of the input
28 */
29 int main(int argc, char* argv[])
30 {
31     int n = atoi(argv[1]); // Get the input number
32
33     printf("%d^2 = %u\n", n, squareOf(n));
34
35     return 0;
36 }

```

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 //Function declaration
5 unsigned int squareOf(int n);
6
7 int main(int argc, char* argv[])
8 {
9     int n = atoi(argv[1]); // Get the input number
10
11     printf("%d^2 = %u\n", n, squareOf(n));
12
13     return 0;
14 }
15
16 // Function definition
17 unsigned int squareOf(int n)
18 {
19     return n*n;
20 }

```

Execution always starts at `main`, and then statements are executed one at a time, in order; when reaching a function call, the execution goes to the function, executing all the statements there, and then returning to the line after the function call. Therefore there is a flow of execution, that needs to be followed. In the second example above, the codes can be put into separate files, namely the header file, where the function declarations are, a file that has the function definition, and a file that has the main function. You must use `#include` to insert the named file into the file that is being compiled, here in both files that include the `main` and `squareOf` functions.

Function arguments are things that you provide to let the function do its job. Communication with functions is through these arguments and return values, if

any. For example, some functions take more than one parameter, like `pow`, which takes two `doubles`, the base and the exponent.

3.8 Glossary

floating-point: A type of variable (or value) that can contain fractions as well as integers. There are a few floating-point types in C++; the one we use in this book is `double`.

initialization: A statement that declares a new variable and assigns a value to it at the same time.

function: A named sequence of statements that performs some useful function. Functions may or may not take parameters, and may or may not produce a result.

parameter: A piece of information you provide in order to call a function. Parameters are like variables in the sense that they contain values and have types.

argument: A value that you provide when you call a function. This value must have the same type as the corresponding parameter.

call: Cause a function to be executed.